arXiv:1806.07857v3 [cs.LG] 10 Sep 2019

RUDDER: Return Decomposition for Delayed Rewards

Jose A. Arjona-Medina* **Michael Gillhofer*** **Michael Widrich***
Thomas Unterthiner **Johannes Brandstetter** **Sepp Hochreiter**†
LIT AI Lab
Institute for Machine Learning
Johannes Kepler University Linz, Austria
† also at Institute of Advanced Research in Artificial Intelligence (IARAI)

Abstract

We propose RUDDER, a novel reinforcement learning approach for delayed rewards in finite Markov decision processes (MDPs). In MDPs the Q-values are equal to the expected immediate reward plus the expected future rewards. The latter are related to bias problems in temporal difference (TD) learning and to high variance problems in Monte Carlo (MC) learning. Both problems are even more severe when rewards are delayed. RUDDER aims at making the expected future rewards zero, which simplifies Q-value estimation to computing the mean of the immediate reward. We propose the following two new concepts to push the expected future rewards toward zero. (i) Reward redistribution that leads to return-equivalent decision processes with the same optimal policies and, when optimal, zero expected future rewards. (ii) Return decomposition via contribution analysis which transforms the reinforcement learning task into a regression task at which deep learning excels. On artificial tasks with delayed rewards, RUDDER is significantly faster than MC and exponentially faster than Monte Carlo Tree Search (MCTS), TD(λ), and reward shaping approaches. At Atari games, RUDDER on top of a Proximal Policy Optimization (PPO) baseline improves the scores, which is most prominent at games with delayed rewards. Source code is available at `https://github.com/ml-jku/rudder` and demonstration videos at `https://goo.gl/EQerZV`.

1 Introduction

Assigning credit for a received reward to past actions is central to reinforcement learning [128]. A great challenge is to learn long-term credit assignment for delayed rewards [65, 59, 46, 106]. Delayed rewards are often episodic or sparse and common in real-world problems [97, 76]. For Markov decision processes (MDPs), the Q-value is equal to the expected immediate reward plus the expected future reward. For Q-value estimation, the expected future reward leads to biases in temporal difference (TD) and high variance in Monte Carlo (MC) learning. For delayed rewards, TD requires exponentially many updates to correct the bias, where the number of updates is exponential in the number of delay steps. For MC learning the number of states affected by a delayed reward can grow exponentially with the number of delay steps. (Both statements are proved after theorems A8 and A10 in the appendix.) An MC estimate of the expected future reward has to average over all possible future trajectories, if rewards, state transitions, or policies are probabilistic. Delayed rewards make an MC estimate much harder.

* authors contributed equally

The main goal of our approach is to construct an MDP that has **expected future rewards equal to zero**. If this goal is achieved, Q-value estimation simplifies to computing the mean of the immediate rewards. To push the expected future rewards to zero, we require two new concepts. The first new concept is **reward redistribution** to create **return-equivalent MDPs**, which are characterized by having the same optimal policies. An optimal reward redistribution should transform a delayed reward MDP into a return-equivalent MDP with zero expected future rewards. However, expected future rewards equal to zero are in general not possible for MDPs. Therefore, we introduce sequence-Markov decision processes (SDPs), for which reward distributions need not to be Markov. We construct a reward redistribution that leads to a return-equivalent SDP with a second-order Markov reward distribution and expected future rewards that are equal to zero. For these return-equivalent SDPs, Q-value estimation simplifies to computing the mean. Nevertheless, the Q-values or advantage functions can be used for learning optimal policies. The second new concept is **return decomposition** and its realization via **contribution analysis**. This concept serves to efficiently construct a proper reward redistribution, as described in the next section. Return decomposition transforms a reinforcement learning task into a regression task, where the sequence-wide return must be predicted from the whole state-action sequence. The regression task identifies which state-action pairs contribute to the return prediction and, therefore, receive a redistributed reward. Learning the regression model uses only completed episodes as training set, therefore avoids problems with unknown future state-action trajectories. Even for sub-optimal reward redistributions, we obtain an enormous speed-up of Q-value learning if relevant reward-causing state-action pairs are identified. We propose RUDDER (RetUrn Decomposition for DElayed Rewards) for learning with reward redistributions that are obtained via return decompositions.

To get an intuition for our approach, assume you repair pocket watches and then sell them. For a particular brand of watch you have to decide whether repairing pays off. The sales price is known, but you have unknown costs, i.e. negative rewards, caused by repair and delivery. The advantage function is the sales price minus the expected immediate repair costs minus the expected future delivery costs. Therefore, you want to know whether the advantage function is positive. — Why is zeroing the expected future costs beneficial? — If the average delivery costs are known, then they can be added to the repair costs resulting in zero future costs. Using your repairing experiences, you just have to average over the repair costs to know whether repairing pays off. — Why is return decomposition so efficient? — Because of pattern recognition. For zero future costs, you have to estimate the expected brand-related delivery costs, which are e.g. packing costs. These brand-related costs are superimposed by brand-independent general delivery costs for shipment (e.g. time spent for delivery). Assume that general delivery costs are indicated by patterns, e.g. weather conditions, which delay delivery. Using a training set of completed deliveries, supervised learning can identify these patterns and attribute costs to them. This is return decomposition. In this way, only brand-related delivery costs remain and, therefore, can be estimated more efficiently than by MC.

Related Work. Our new learning algorithm is gradually changing the reward redistribution during learning, which is known as shaping [120, 128]. In contrast to RUDDER, potential-based shaping like reward shaping [87], look-ahead advice, and look-back advice [144] use a fixed reward redistribution. Moreover, since these methods keep the original reward, the resulting reward redistribution is not optimal, as described in the next section, and learning can still be exponentially slow. A monotonic positive reward transformation [91] also changes the reward distribution but is neither assured to keep optimal policies nor to have expected future rewards of zero. Disentangled rewards keep optimal policies but are neither environment nor policy specific, therefore can in general not achieve expected future rewards being zero [28]. Successor features decouple environment and policy from rewards, but changing the reward changes the optimal policies [7, 6]. Temporal Value Transport (TVT) uses an attentional memory mechanism to learn a value function that serves as fictitious reward [59]. However, expected future rewards are not close to zero and optimal policies are not guaranteed to be kept. Reinforcement learning tasks have been changed into supervised tasks [108, 8, 112]. For example, a model that predicts the return can supply update signals for a policy by sensitivity analysis. This is known as "backpropagation through a model" [86, 101, 102, 142, 111, 4, 5]. In contrast to these approaches, (i) we use contribution analysis instead of sensitivity analysis, and (ii) we use the whole state-action sequence to predict its associated return.

2 Reward Redistribution and Novel Learning Algorithms

Reward redistribution is the main new concept to achieve expected future rewards equal to zero. We start by introducing MDPs, return-equivalent sequence-Markov decision processes (SDPs), and reward redistributions. Furthermore, optimal reward redistribution is defined and novel learning algorithms based on reward redistributions are introduced.

MDP Definitions and Return-Equivalent Sequence-Markov Decision Processes (SDPs). A finite Markov decision process (MDP) $\mathcal{P}$ is 5-tuple $\mathcal{P} = (\mathcal{S}, \mathcal{A}, \mathcal{R}, p, \gamma)$ of finite sets $\mathcal{S}$ of states s (random variable S_t at time t), $\mathcal{A}$ of actions a (random variable A_t), and $\mathcal{R}$ of rewards r (random variable R_{t+1}). Furthermore, $\mathcal{P}$ has transition-reward distributions $p(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a)$ conditioned on state-actions, and a discount factor $\gamma \in [0, 1]$. The marginals are $p(r \mid s, a) = \sum_{s'} p(s', r \mid s, a)$ and $p(s' \mid s, a) = \sum_r p(s', r \mid s, a)$. The expected reward is $r(s, a) = \sum_r r p(r \mid s, a)$. The return G_t is $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$, while for finite horizon MDPs with sequence length T and $\gamma = 1$ it is $G_t = \sum_{k=0}^{T-t} R_{t+k+1}$. A Markov policy is given as action distribution $\pi(A_t = a \mid S_t = s)$ conditioned on states. We often equip an MDP $\mathcal{P}$ with a policy π without explicitly mentioning it. The action-value function $q^\pi(s, a)$ for policy π is $q^\pi(s, a) = \mathrm{E}_\pi [G_t \mid S_t = s, A_t = a]$. The goal of learning is to maximize the expected return at time $t = 0$, that is $v_0^\pi = \mathrm{E}_\pi [G_0]$. The optimal policy π^* is $\pi^* = \operatorname{argmax}_\pi [v_0^\pi]$. A *sequence-Markov decision process* (SDP) is defined as a decision process which is equipped with a Markov policy and has Markov transition probabilities but a reward that is not required to be Markov. Two SDPs $\tilde{\mathcal{P}}$ and $\mathcal{P}$ are *return-equivalent* if (i) they differ only in their reward distribution and (ii) they have the same expected return at $t = 0$ for each policy π: $\tilde{v}_0^\pi = v_0^\pi$. They are *strictly return-equivalent* if they have the same expected return for every episode and for each policy π. Strictly return-equivalent SDPs are return-equivalent. Return-equivalent SDPs have the same optimal policies. For more details see Section A2.2 in the appendix.

Reward Redistribution. Strictly return-equivalent SDPs $\tilde{\mathcal{P}}$ and $\mathcal{P}$ can be constructed by reward redistributions. A *reward redistribution* given an SDP $\tilde{\mathcal{P}}$ is a procedure that redistributes for each sequence $s_0, a_0, \ldots, s_T, a_T$ the realization of the sequence-associated return variable $\tilde{G}_0 = \sum_{t=0}^{T} \tilde{R}_{t+1}$ or its expectation along the sequence. Later we will introduce a reward redistribution that depends on the SDP $\tilde{\mathcal{P}}$. The reward redistribution creates a new SDP $\mathcal{P}$ with the redistributed reward R_{t+1} at time $(t+1)$ and the return variable $G_0 = \sum_{t=0}^{T} R_{t+1}$. A reward redistribution is second order Markov if the redistributed reward R_{t+1} depends only on $(s_{t-1}, a_{t-1}, s_t, a_t)$. If the SDP $\mathcal{P}$ is obtained from the SDP $\tilde{\mathcal{P}}$ by reward redistribution, then $\tilde{\mathcal{P}}$ and $\mathcal{P}$ are strictly return-equivalent. The next theorem states that the optimal policies are still the same for $\tilde{\mathcal{P}}$ and $\mathcal{P}$ (proof after Section Theorem S2).

Theorem 1. *Both the SDP $\tilde{\mathcal{P}}$ with delayed reward $\tilde{R}_{t+1}$ and the SDP $\mathcal{P}$ with redistributed reward R_{t+1} have the same optimal policies.*

Optimal Reward Redistribution with Expected Future Rewards Equal to Zero. We move on to the main goal of this paper: to derive an SDP via reward redistribution that has expected future rewards equal to zero and, therefore, no delayed rewards. At time $(t-1)$ the immediate reward is R_t with expectation $r(s_{t-1}, a_{t-1})$. We define the expected future rewards $\kappa(m, t-1)$ at time $(t-1)$ as the expected sum of future rewards from R_{t+1} to R_{t+1+m}.

Definition 1. *For $1 \leqslant t \leqslant T$ and $0 \leqslant m \leqslant T - t$, the expected sum of delayed rewards at time $(t-1)$ in the interval $[t+1, t+m+1]$ is defined as $\kappa(m, t-1) = \mathrm{E}_\pi [\sum_{\tau=0}^{m} R_{t+1+\tau} \mid s_{t-1}, a_{t-1}]$.*

For every time point t, the expected future rewards $\kappa(T-t-1, t)$ given (s_t, a_t) is the expected sum of future rewards until sequence end, that is, in the interval $[t+2, T+1]$. For MDPs, the Bellman equation for Q-values becomes $q^\pi(s_t, a_t) = r(s_t, a_t) + \kappa(T-t-1, t)$. We aim to derive an MDP with $\kappa(T-t-1, t) = 0$, which gives $q^\pi(s_t, a_t) = r(s_t, a_t)$. In this case, learning the Q-values simplifies to estimating the expected immediate reward $r(s_t, a_t) = \mathrm{E}[R_{t+1} \mid s_t, a_t]$. Hence, the reinforcement learning task reduces to computing the mean, e.g. the arithmetic mean, for each state-action pair (s_t, a_t). A reward redistribution is defined to be *optimal*, if $\kappa(T-t-1, t) = 0$ for $0 \leqslant t \leqslant T-1$. In general, an optimal reward redistribution violates the Markov assumptions and the Bellman equation does not hold (proof after Theorem A3 in the appendix). Therefore, we